DEVELOPING A RAPID URBAN FOREST ASSESSMENT SYSTEM FOR SUSTAINABLE CITY GREENIFICATION

A Thesis

presented to

the Faculty of California Polytechnic State University,

San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Computer Science

by

Daniel Gonzalez

June 2026

COMMITTEE MEMBERSHIP

TITLE: Developing a Rapid Urban Forest Assessment System for Sustainable City Greenification

AUTHOR: Daniel Gonzalez

DATE SUBMITTED: June 2026

COMMITTEE CHAIR: Dr. Alex Dekhtyar, Ph.D.
Professor of Computer Science

COMMITTEE MEMBER: Professor Matt Ritter, Ph.D.
Professor of Biological Sciences

COMMITTEE MEMBER: Jenn Yost, Ph.D.
Professor of Biological Sciences

COMMITTEE MEMBER: Professor Jonathan Ventura, Ph.D.
Professor of Computer Science

ABSTRACT

Developing a Rapid Urban Forest Assessment System for Sustainable City Greenification

Daniel Gonzalez

This thesis presents the Rapid Urban Forest Assessment (RUFA) system, a web-based platform that integrates urban tree inventories and aerial tree detection to assess forest health across California's census-designated places. RUFA combines inventoried tree records with coordinates detected from high-resolution multispectral imagery using convolutional neural networks, then computes a composite RUFA Score from four metrics: canopy cover percentage, trees per capita, tree diversity (TD-50), and tree evenness. The thesis addresses two engineering challenges in building the dashboard: querying and aggregating over seven million tree records in real time, and rendering spatial summaries at multiple zoom levels without recomputing cluster assignments on every map interaction. A MySQL schema with indexing and cascading materialized views enables sub-second city-level queries, and a SuperCluster index paired with Voronoi tessellation supports interactive map visualization. Deployed on AWS, RUFA provides urban foresters, planners, and researchers with comparable scores and multi-scale map outputs to support evidence-based urban forestry management.

Keywords: urban forestry, tree density, biodiversity, TD-50, scalable, convolutional neural networks

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to the following individuals for their invaluable support and contributions to the development of the RUFA System:

- **Professor Matt Ritter** and **Jenn Yost**, for entrusting me with the UFEI infrastructure and RUFA project. Their confidence and guidance have been instrumental in the success of this research.
- **Dr. Alex Dekhtyar**, for his continuous guidance and insightful feedback throughout the project. His expertise has significantly shaped the direction and quality of this research.
- **Professor Rosa Chang**, **Cameron Squire**, **Dylan Lewis**, **Eric Tong**, **Kristian Markov**, and **Katherin Clark** from San Jose State University, for their dedicated efforts in developing the front-end of RUFA. Their efforts greatly enhanced the functionality and user experience of the assessment system.
- **Cami Pawlak**, for her assistance in providing RUFA data and related formulas, which was essential for the implementation and testing phases of the project.

Additionally, I would like to acknowledge all other individuals and organizations that have supported me throughout this research and development journey.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

Chapter 1

INTRODUCTION

Urban and community forestry (U&CF) programs play a crucial role in nurturing and enhancing urban green spaces, contributing significantly to the environmental, social, and economic well-being of urban communities. To support these initiatives, the Urban Forest Ecosystems Institute (UFEI) at Cal Poly develops tools to evaluate and report on U&CF activities across California. The Rapid Urban Forest Assessment (RUFA) system is UFEI's web-based platform for integrating, analyzing, and disseminating comprehensive urban forestry data. RUFA serves as a centralized hub for data collection and analysis, providing actionable insights into the health and management of urban green spaces while integrating resources with other UFEI products and services, such as the California tree identification portal SelecTree (selectree.calpoly.edu).

Comparable measurement remains difficult at statewide scale. California school campuses, for example, have only 6.4% median tree canopy in student zones [8], and about 85% of urban public elementary schools lost some canopy between 2018 and 2022 [25]. RUFA responds by turning inventory and aerial-detection records into a composite score and multi-scale maps for California census-designated places.

This thesis addresses two engineering problems encountered in building RUFA. The system must query and aggregate over seven million tree records across hundreds of California census-designated places in real time, yet naive storage and retrieval strategies produce multi-second latencies unsuitable for an interactive dashboard. It must also render spatial summaries at multiple zoom levels without recomputing

cluster assignments on every pan or zoom event. Meeting these requirements drove the database design choices in chapter 3 and the clustering work in chapter 5.

The specific contributions of this thesis are as follows:

- A MySQL relational schema with indexing and materialized views for sub-second city-level queries over RUFA's tree inventory, aerial-detection, and geographic boundary data.
- A cascading materialized-view architecture that pre-computes and propagates RUFA Score metrics across census places, ZIP codes, tracts, and counties.
- Evaluation of spatial indexing strategies for assigning tree points to administrative polygons at RUFA scale.
- Progressive clustering algorithms, from K-means to SuperClustering with Voronoi diagrams, for multi-scale map visualization.
- An AWS three-tier deployment compatible with the existing SelecTree stack, summarized as supporting infrastructure in chapter 4.

The rest of this document is organized as follows. chapter 2 establishes UFEI context, the RUFA Score, and the inventory and detection data sources. chapter 3 covers spatial indexing, schema design, and materialized views. chapter 4 summarizes deployment. chapter 5 develops the map-clustering pipeline. chapter 6 summarizes the contributions and discusses future work.

Chapter 2

BACKGROUND

This Chapter establishes the context for the RUFA system. section 2.1 introduces the Urban Forest Ecosystems Institute and its related tools. section 2.2 describes RUFA as a software application, its core use cases, and its data sources. section 2.3 defines the composite score early, then section 2.4 and section 2.5 provide supporting ecological context for the score components.

2.1 Urban Forest Ecosystems Institute

The Urban Forest Ecosystems Institute (UFEI) at Cal Poly addresses the growing need for improved management of California's urban forests. UFEI develops and maintains a suite of public-facing tools and research resources that support tree selection, identification, spatial analysis, and urban forest assessment across the state. These include SelecTree[1], an online tree selection guide covering hundreds of species; the Urban Tree Key[2] for field identification; California Big Trees[3], which documents champion trees statewide; and the Urban Tree Map[4], a spatial analysis platform for California's inventoried urban trees. UFEI also supports applied research projects that apply remote sensing, machine learning, and ecological modeling to urban forestry questions.

[1] https://selectree.calpoly.edu

[2] https://urbantreekey.calpoly.edu

[3] https://californiabigtrees.calpoly.edu/

[4] https://ucfmtm.calpoly.edu

UFEI's mission is to connect scientific research with practical tools that urban foresters, planners, and community members can use to make informed decisions about tree planting, maintenance, and policy. The institute collaborates with municipal agencies, nonprofit organizations, and private arborist firms to aggregate urban tree inventory data, and partners with researchers at Cal Poly and beyond to develop new methods for measuring urban forest structure, diversity, and ecosystem services. Projects supported by UFEI span species-diversity assessment, urban heat mitigation, canopy mapping, and automated tree detection from aerial imagery.

The Rapid Urban Forest Assessment (RUFA) is one of UFEI's flagship assessment tools. It was conceived to give California communities a standardized, data-driven view of their urban forest health by combining the institute's extensive inventory compilations with automated tree detection from high-resolution aerial imagery. Where earlier UFEI resources provided species-level information or static inventory maps, RUFA aggregates these data sources into a single dashboard that computes comparable metrics and composite scores for every census-designated place in the state.

2.2 RUFA System Overview

RUFA is a read-only web dashboard designed for urban foresters, planners, and researchers who need to compare forest conditions across California communities without managing underlying inventory data. Its core use cases are as follows: a user selects a city or town and views an overview page showing the RUFA Score and component statistics; the user explores an interactive map of detected and inventoried trees, zooming in and out to review density and diversity patterns at city, ZIP-code, and tract levels; and the user compares scores across places to identify communities

that outperform or lag statewide norms. Figure 2.1 illustrates the map view, which supports multi-scale exploration of tree density within a selected city.

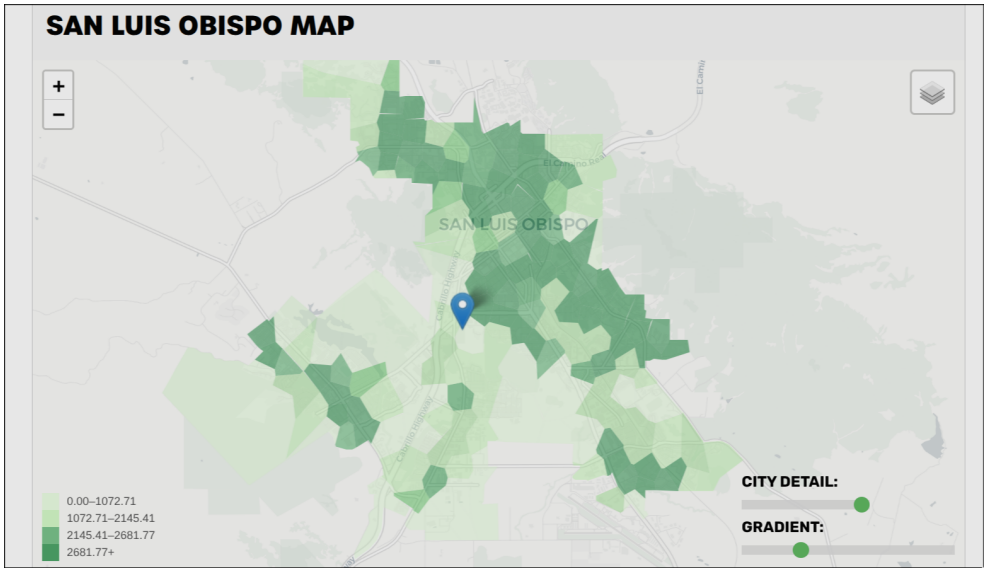


Figure 2.1: *Interactive map view in RUFA showing tree density by Voronoi cell for San Luis Obispo.*

RUFA integrates two primary data sources to provide comprehensive insights into urban forest ecosystems. The first data source offers highly detailed tree characteristics, derived from an extensive compilation of California urban forest inventories. These inventories were aggregated from diverse sources, including private arborist firms (West Coast Arborists, Davey Tree Company, APlus Tree Care) and public entities such as CAL FIRE [13]. This robust dataset ensures a high level of granularity and accuracy in tree attribute information, including species, genus, family, structural measurements, and per-tree canopy estimates.

The second data source supplies tree coordinates detected through a deep learning pipeline developed by Ventura et al. [26]. This method employs high-resolution multispectral aerial imagery to identify individual trees in urban settings; a convolutional neural network generates a confidence map by regressing detected tree locations, en-

abling precise spatial quantification of urban tree coverage in areas where formal inventory data are incomplete or absent.

Together, these sources allow RUFA to report metrics—canopy cover, trees per capita, species diversity, and evenness—at the census-place, ZIP-code, and tract levels, and to compute a composite RUFA Score developed at UFEI that supports side-by-side comparison of urban forest performance across California communities. The RUFA application implements its own metric for diversity and health of the urban tree cover, described in section 2.3.

The two sources are complementary. Inventory alone is species-rich but unevenly covered; detection alone is spatially broad but carries no taxonomy. Combined, RUFA can still compute spatial metrics statewide and attach diversity metrics wherever inventory overlaps those polygons. The join is operationalized through point-in-polygon assignment to place, ZIP, and tract boundaries (chapter 3).

2.3 RUFA Score

The Rapid Urban Forest Assessment (RUFA) system provides a composite "RUFA Score" for each census-designated place in California, facilitating an at-a-glance evaluation of urban forest health and sustainability. This score integrates four equally weighted metrics:

- **Canopy cover percentage (CCP):** Reflects the proportion of land area covered by tree canopy.
- **Trees per capita (TPC):** Indicates tree distribution relative to population size.

- **Tree diversity (TD):** Incorporates the TD-50 index [13] to capture species richness and dominance.
- **Tree evenness (TE):** Reflects how uniformly trees are distributed spatially within a place.

Each component metric is computed within RUFA from the data sources described above, then *binned* into a score of 1 to 5 based on its distribution across California. Summing these binned values yields a maximum of 20 possible points. The final RUFA Score is then calculated as:

$$\text{RUFA Score} = \left(\frac{\text{CCP} + \text{TD} + \text{TE} + \text{TPC}}{20} \right) \times 100. \tag{2.1}$$

The RUFA Score ranges from 0 to 100, with higher values (closer to 100) indicating stronger overall performance.

Below, we discuss each of the constituent parts of the RUFA Score.

Canopy cover percentage. CCP measures the percentage of land area covered by tree canopy within each census-designated place. The data come from EarthDefine's 2018 California canopy product, purchased for the state by the U.S. Forest Service and distributed as open data [5, 22]. The dataset reports 96.6% accuracy across the United States. Because EarthDefine's urban boundaries do not align perfectly with census-designated place boundaries, RUFA computes canopy percentages only where EarthDefine coverage overlaps a place polygon, but reports the value for the full place boundary; places with no overlapping canopy data receive a score of NA. Canopy cover percentage is a widely used urban forestry metric in California and supports measurement against the California Urban Forestry Act of 1978 goal of a 10-percent

increase in urban tree canopy cover by 2035, with priority for disadvantaged and low-income communities [20].

Trees per capita. TPC divides the total tree count within a geographic area by its human population. Tree counts combine inventoried records from the California Urban Forest Inventory with coordinates detected by the Ventura et al. pipeline, deduplicated and assigned to the appropriate place, ZIP, or tract polygon. Population figures come from U.S. Census Bureau data stored in the RUFA database (`human_population` per city/place), including 2020 Decennial Census figures [21], and are propagated to associated geographic units.

Tree diversity. The TD component applies the TD-50 index defined in section 2.4. RUFA computes species counts from the urban forest inventory records assigned to each geographic area and evaluates Equation 2.2 over those counts. No external diversity index is imported; the value is derived entirely from inventory species-abundance data at query or materialized-view refresh time.

Tree evenness. TE measures how evenly tree density is distributed across a census-designated place, reflecting how consistently residents experience tree access. To compute it, the place is subdivided into census blocks; trees in each block are counted and converted to a block-level density in trees per square kilometer. The standard deviation of these block-level densities is the tree evenness score. Higher values indicate less uniform distribution: a score of 250, for instance, means that census blocks typically fall about 250 trees per km^2 above or below the city-wide average density.

By normalizing each component and summing their contributions, the RUFA Score offers a standardized, comparable index of urban forest health. This approach ensures

that areas with varying population densities, canopy coverage, and species compositions can be evaluated on a consistent scale, guiding strategic planning and policy decisions to enhance urban forest resilience.

2.4 Diversity Metrics

Diversity metrics play a crucial role in understanding the composition and resilience of urban forests. Among these, the TD-50 index, introduced by Love et al. [13], has emerged as a pivotal tool for assessing urban tree species diversity. This index identifies the minimum number of species needed to account for the top 50% of urban forest abundance, providing a snapshot of both ecological dominance and species richness.

Formally, let n_i denote the number of trees of species i in a geographic area, with total tree count $N = \sum_i n_i$. The relative abundance of species i is $p_i = n_i/N$. Species are ranked in descending order of n_i, and the TD-50 index is the smallest integer k such that the cumulative relative abundance of the top k species reaches at least 50%:

$$\text{TD-50} = \min \left\{ k \;\middle|\; \sum_{i=1}^{k} p_{(i)} \geq 0.5 \right\}, \tag{2.2}$$

where $p_{(i)}$ denotes the relative abundance of the ith most abundant species. Higher TD-50 values indicate that more species are required to reach half of the tree population, reflecting greater diversity and reduced dependence on a few dominant species. RUFA computes TD-50 from species counts in the urban forest inventory assigned to each geographic area.

The value of such metrics lies in their ability to guide urban forestry programs toward enhanced biodiversity. High diversity levels improve resilience to pests, diseases,

and environmental changes, while low diversity often signifies vulnerability. For instance, McPherson et al. [14] highlight the importance of diversifying tree species as a strategy to bolster urban forest resilience against climate change impacts. Furthermore, Livesley et al. [11] emphasize the ecological functions provided by diverse urban forests, such as improved air quality, habitat provision, and the regulation of urban microclimates.

Research underscores that species diversity influences urban forests' ability to adapt to environmental stressors, including extreme weather events and urban heat islands. Therefore, incorporating diversity metrics into urban forestry management is critical for maintaining ecological stability and sustainability.

2.5 Canopy and Density

Urban tree canopy coverage and density are fundamental indicators of urban forest health and performance. Studies like those by Livesley et al. [11] and Sanusi et al. [19] underscore the significance of these metrics in understanding urban tree contributions to ecological and social well-being.

Canopy coverage is particularly effective in regulating urban temperatures and reducing solar radiation. This is especially relevant for small-statured trees, which Sanusi et al. [19] found to be instrumental in urban cooling, despite their limited size. Additionally, canopy density directly correlates with urban areas' capacity to manage stormwater runoff, as shown in Xiao and McPherson's [27] research.

Density metrics also provide insights into forest sustainability. Higher tree densities often indicate robust forest growth and effective carbon sequestration capabilities. However, overcrowding can lead to competition for resources, reducing overall health

and resilience. Balancing canopy coverage and density is therefore essential for maximizing the benefits of urban forests, from cooling effects and stormwater management to improving urban aesthetics and human well-being.

These considerations directly motivated two RUFA design decisions. Canopy cover percentage and trees per capita are core inputs to the RUFA Score (section 2.3), and the density of tree points across the map drove the development of the hierarchical clustering algorithms described in chapter 5. RUFA therefore treats canopy and density not as abstract ecological ideals but as measurable quantities that the system computes, stores, and surfaces to users through the dashboard.

Chapter 3

RUFA DATABASE DESIGN

In this Chapter, we delve into the rationale behind the relational database for the RUFA project, a system designed to track tree records across various geographic and ecological dimensions. A relational database provides an ideal solution for managing structured data, such as tree attributes, geographic metrics, and hierarchical relationships between places, zip codes, and tracts. We will explore the MySQL workflow, emphasizing data integrity, normalization principles, materialized views for efficient querying, and indexing strategies to optimize database performance. These features ensure scalable, accurate, and high-performance data management for the RUFA project.

At production scale, the workload includes on the order of 7.2 million detector coordinates and tens of millions of inventory-related rows across 702+ places, with interactive dashboard reads targeting well under 200 ms. Before a score can be served, each tree point must be assigned to the correct administrative polygons—a two-dimensional locality problem for which independent one-dimensional indexes are a poor fit [18].

3.1 Proximity Point Search

3.1.1 Baseline Geospatial Filter

The most intuitive proximity search pattern is to draw a bounding box around a target coordinate and apply a distance-constrained `WHERE` clause. Conceptually, this

is straightforward and easy to explain in user-facing tools. However, at RUFA scale (tens of millions of rows), the naive implementation can degrade into near table-scan behavior when the engine cannot prune enough records early in execution.

3.1.2 Why Separate Latitude/Longitude Indexes Plateau

Ordinary B-tree indexes on latitude or longitude alone improve one dimension at a time. A latitude range becomes a horizontal band; longitude then filters that band after many candidates remain. Dense urban inventories make that residual set large enough that interactive dashboards feel unresponsive. Spatial indexes encode both dimensions in one access path [18].

3.1.2.1 Hash-Based Spatial Partitioning

Hash-based approaches (for example fixed grids, geohash variants [16], or Cartesian tiering) divide the map into smaller cells and assign records to cell keys. In plain terms, the earth is bucketed into many small boxes, and queries first retrieve matching buckets before testing exact geometry. This is often simple to implement and scales well for coarse filtering.

The tradeoff is that cell quality depends heavily on resolution choice. If cells are too large, many unrelated trees collapse into the same bucket and candidate sets grow quickly. If cells are too small, index management and boundary effects increase. Another limitation for RUFA is geometric fidelity: polygon membership and boundary precision are approximated by the selected grid resolution before exact tests are applied.

3.1.2.2 Tree-Based Spatial Indexing

Tree-based methods preserve spatial hierarchy by recursively partitioning space and are generally better suited for variable density data:

Quadtree. Recursively splits 2D space into four quadrants [6]. It is simple and effective for many point-heavy workloads, but balance and depth can vary widely with uneven urban density.

Google S2. Projects the globe onto hierarchical cells over a cube-based decomposition [24]. It is robust for global-scale consistency and spherical geometry, especially when cross-region precision matters.

R-tree. Organizes geometry by nested minimum bounding rectangles (MBRs), enabling efficient pruning of large regions that cannot contain a match [9]. This is particularly effective for polygon layers and mixed point/polygon workflows.

3.1.3 RUFA Approach: R-tree Candidate Pruning Plus Exact Geometry

RUFA uses an R-tree-oriented workflow for polygon layers such as place, ZIP, and tract boundaries. At index-build time, each polygon's MBR is inserted into the tree as a searchable envelope [9]. During lookup, a point query retrieves a very small set of nearest or intersecting envelope candidates, and then runs an exact geometry containment test against the candidate polygons.

This two-phase strategy is important: the tree handles fast candidate pruning, while exact polygon containment preserves correctness at boundaries. In other words,

RUFA avoids scanning every row while still returning topologically accurate geographic assignments. These proximity-search requirements directly shape the relational design choices in RUFA, including how tables are normalized, how keys connect geographic entities, and where indexes are introduced for predictable query performance.

3.2 SQL Workflow

With that spatial-query context established, a well-defined SQL workflow is crucial for maintaining data integrity and ensuring efficient data retrieval [4]. The following Sections detail the tables used in the RUFA database, the normalization process adopted, and the overall database schema design.

3.2.1 Tables and Normalization

The RUFA database consists of several tables designed to minimize redundancy and dependency. By normalizing the database to the third normal form (3NF), we ensure that each table contains data related to a specific topic, thereby enhancing data consistency and simplifying maintenance.

3.2.2 Entity Relationship Diagram

Figure 3.1 illustrates the ERD for the RUFA database schema. This diagram highlights the relationships between different entities, such as Points, Polygons, and Places, ensuring that the database structure supports all necessary operations and queries efficiently. Points are derived from our core data sources, while polygons represent geographic boundaries including census places, ZIP codes, and census tracts.

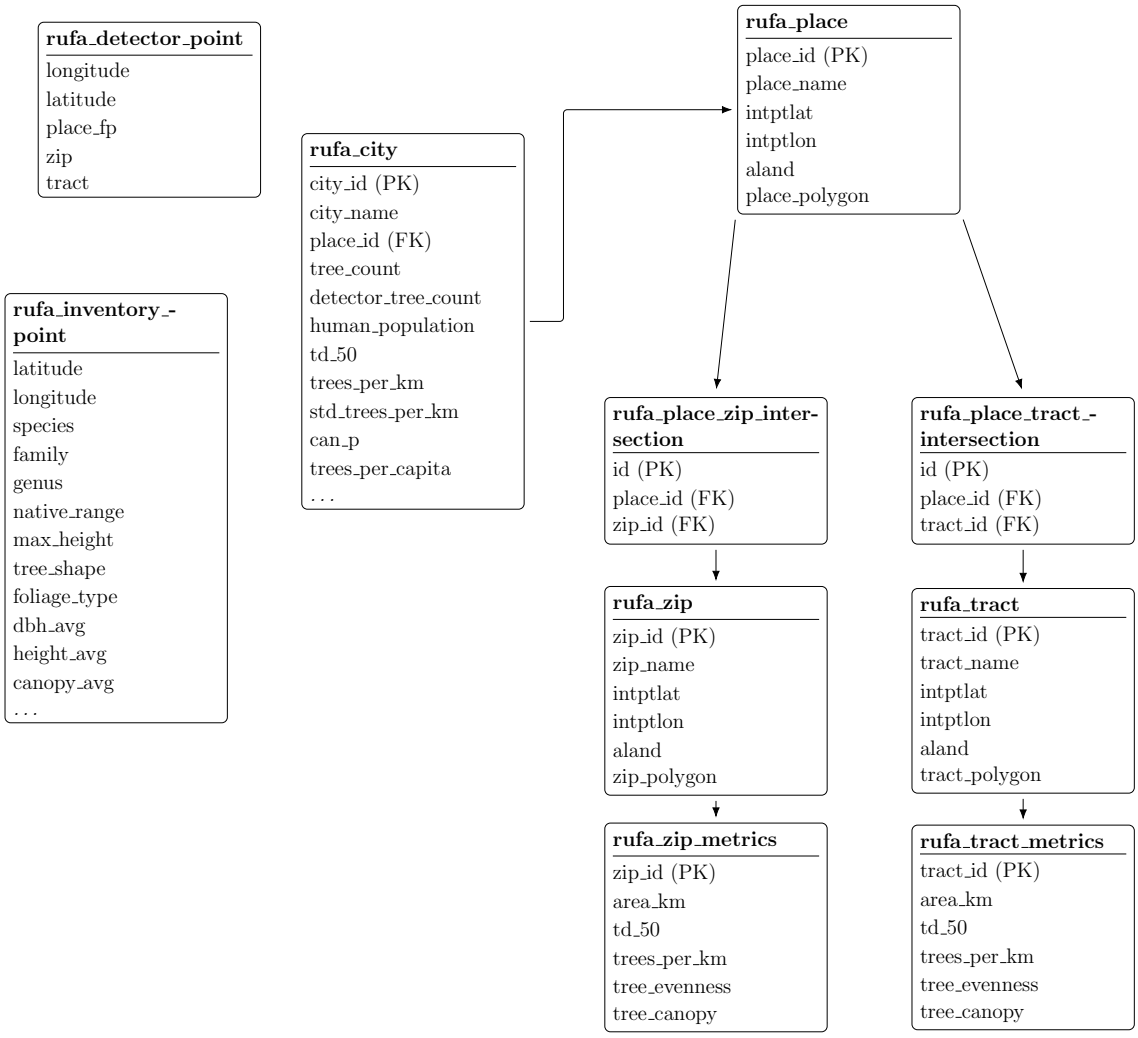


Figure 3.1: *Entity Relationship Diagram (ERD) for the RUFA database schema*

3.2.3 Alternative Database Solutions

While MySQL serves as the database choice for the RUFA, there are several alternative systems that offer enhanced capabilities for spatial data management and analysis. Given that the RUFA project relies on precise geographical information, including census tracts, city boundaries, and tree coordinates, optimizing spatial queries and performance can be a critical consideration.

PostgreSQL/PostGIS. One of the most commonly cited alternatives for spatially intensive applications is PostgreSQL coupled with its PostGIS extension [17]. PostGIS extends the relational engine to include advanced geospatial data types, indexing methods (e.g., GiST, SP-GiST), and an extensive library of geospatial functions (for instance, ST_Intersect, S_Buffer, and ST_Distance). These features support complex spatial queries, such as determining which trees fall within a specific bounding polygon or calculating nearest-neighbor distances between tree points. Additionally, PostGIS's robust handling of coordinate reference systems (CRS) and transformations can greatly simplify cross-regional comparisons within the RUFA framework.

MongoDB with Geospatial Indexes. MongoDB, a NoSQL document database, also offers geospatial indexing that can handle queries for distance-based searches on points, lines, and polygons [3]. This schema-free approach can be advantageous for storing heterogeneous tree data, especially when species attributes vary significantly, since new fields can be added without redefining a rigid schema.

GIS-Specific Databases. Some organizations adopt specialized spatial database solutions, such as *Esri's ArcGIS Enterprise* or *Oracle Spatial and Graph*, for high-volume or enterprise-level spatial data management. These systems offer robust toolsets for spatial analyses, but they can come with higher licensing costs and steeper learning curves. For smaller projects or open-source-driven initiatives, this may pose resource constraints and limit broad community involvement.

Why MySQL Remains in Use. Despite these more spatially advanced alternatives, MySQL remains the primary choice for the RUFA project due to its historical ties with the selectree.calpoly.edu infrastructure. Many existing scripts, tools, and user-facing applications are already deeply integrated with MySQL, making a com-

plete or partial transition to another platform expensive in terms of development effort and time. Moreover, while MySQL's native spatial features are comparatively limited, they can still manage a variety of spatial queries by leveraging *SPATIAL indexes* on MyISAM or InnoDB tables.

For MyISAM and InnoDB tables, search operations in columns containing spatial data can be optimized using SPATIAL indexes. The most typical operations are:

- *Point queries* that search for all objects containing a given point
- *Region queries* that search for all objects overlapping a specified region

MySQL uses *R-Trees* with quadratic splitting for SPATIAL indexes on geometry columns. A SPATIAL index is built using the *minimum bounding rectangle* (MBR) of a geometry. For most geometries, the MBR is the smallest rectangle that completely encloses the shape. For a horizontal or vertical linestring, the MBR degenerates to that linestring; and for a point, it degenerates to a single point. Spatial predicates on full `GEOMETRY` values (for example, point-in-polygon or intersection tests) are evaluated using these `SPATIAL` indexes; a conventional B-tree index is not the mechanism used to index the geometry column itself for that style of query. When applications need fast relational access to *scalar* values derived from geometry (such as individual coordinates or numeric projections), the usual pattern is to store those values in stored generated columns or, in MySQL 8.0.13 and later, to define functional key parts on expressions, and then apply ordinary B-tree indexes to those generated or expression columns, rather than to the raw geometry column.

Ultimately, although PostgreSQL/PostGIS/MongoDB has geospatial indexes, or specialized GIS databases may outperform MySQL in certain spatial workflows, the RUFA project's current operational environment and alignment with Selectree's ar-

chitecture underscore MySQL's continued use. Future iterations of RUFA could incorporate or migrate to these alternatives if spatial demands grow significantly, for example by leveraging hybrid architectures that pair MySQL's relational strengths with specialized spatial extensions in a multi-database ecosystem.

3.3 Materialized Views

Materialized views are used in the RUFA database to store the results of complex queries, thereby improving query performance. To ensure that these views remain up-to-date with the underlying data, we implement automatic refresh mechanisms. This can be achieved using database triggers or scheduled jobs that periodically update the materialized views based on changes in the base tables.

3.4 Indexing Strategies

Effective indexing is essential for optimizing query performance in the RUFA database. By carefully selecting which columns to index, we can significantly reduce query execution time. Automatic updates to indexes are managed by the database system, which maintains the indexes as data is inserted, updated, or deleted. Additionally, regular index maintenance tasks, such as rebuilding fragmented indexes, are scheduled to ensure optimal performance.

3.4.1 Automatic Index Maintenance

Modern relational database management systems (RDBMS) handle index maintenance automatically. However, for large databases like RUFA, it's advisable to monitor index performance and perform manual maintenance when necessary. Tools and

scripts can be employed to automate these maintenance tasks, ensuring that indexes remain efficient without manual intervention.

3.5 Materialized View Implementation for RUFA Score Management

3.5.1 Overview

The RUFA system relies on complex calculations involving multiple metrics (canopy cover percentage (CCP), trees per capita (TPC), tree diversity (TD-50), and tree evenness (TE)) to generate composite RUFA scores for census-designated places across California. Given the computational intensity of these calculations and the need for responsive user queries, implementing materialized views becomes essential for maintaining system performance while ensuring data accuracy.

The system faces a unique challenge in that geographic areas are hierarchically related: when tree data changes in a city like Los Angeles, the system must update not only the city-level statistics but also all associated ZIP codes, census tracts, climate zones, and county-level aggregations. This cascading update requirement necessitates a sophisticated materialized view strategy that can efficiently propagate changes through the geographic hierarchy.

3.5.2 Geographic Cascade Architecture

Since MariaDB does not natively support true materialized views like PostgreSQL, the RUFA system implements a specialized architecture using regular tables that function as materialized views, combined with automated refresh mechanisms to maintain data consistency across geographic hierarchies.

The core innovation lies in the cascading update system that recognizes the interconnected nature of geographic boundaries. When tree inventory data changes in any location, the system automatically identifies and updates all related geographic areas through a relationship mapping table and stored procedure framework. This ensures that statistics remain synchronized across ZIP codes, cities, census tracts, climate zones, and counties without requiring manual intervention.

3.5.2.1 Unified Statistics Table

The system maintains a central `tree_stats_by_location` table that stores pre-calculated metrics for all geographic location types within a single, normalized structure. This table contains comprehensive tree statistics including species diversity counts, average tree characteristics, native species distributions, and water requirement breakdowns. Each record is identified by a location type (zip, city, tract) and corresponding location code, enabling efficient querying across different geographic scales.

3.5.2.2 Relationship Mapping and Cascade Logic

A separate `location_relationships` table maintains the hierarchical connections between different geographic boundaries, enabling the system to determine which areas need updates when tree data changes. The cascade logic is implemented through stored procedures that automatically refresh statistics for all related geographic areas when triggered by data modifications.

For example, when tree inventory data changes in Los Angeles, the system automatically updates statistics for the city itself, all ZIP codes within Los Angeles boundaries, associated census tracts, the relevant climate zone, and Los Angeles County. This

ensures consistency across all geographic scales without requiring users or administrators to manually trigger updates for each affected area.

Trigger-Based Updates Database triggers are implemented on core tables to automatically invalidate and schedule refreshes of dependent materialized views:

Code Listing 3.1: *Database trigger for automatic refresh scheduling*

```
DELIMITER $
CREATE TRIGGER update_rufa_scores_trigger
AFTER INSERT ON tree_inventory
FOR EACH ROW
BEGIN
    UPDATE mv_refresh_schedule
    SET needs_refresh = TRUE,
        last_modified = NOW()
    WHERE view_name = 'mv_rufa_scores'
    AND place_id = NEW.place_id;
END$
DELIMITER ;
```

Scheduled Batch Processing A scheduled job runs every N hours to process bulk updates and recalculate scores for modified areas:

Code Listing 3.2: *Batch refresh procedure for RUFA scores*

```
-- Batch refresh procedure
DELIMITER $
CREATE PROCEDURE RefreshRUFAScores()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE place_to_update VARCHAR(50);

    -- Cursor for places needing updates
    DECLARE place_cursor CURSOR FOR
        SELECT DISTINCT place_id
        FROM mv_refresh_schedule
        WHERE needs_refresh = TRUE;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done =
       TRUE;

    OPEN place_cursor;

    refresh_loop: LOOP
        FETCH place_cursor INTO place_to_update;
        IF done THEN
            LEAVE refresh_loop;
        END IF;

        -- Recalculate RUFA score components
        CALL CalculateRUFAScore(place_to_update);

        -- Mark as refreshed
        UPDATE mv_refresh_schedule
        SET needs_refresh = FALSE,
            last_refreshed = NOW()
        WHERE place_id = place_to_update;

    END LOOP;

    CLOSE place_cursor;
END$
DELIMITER ;
```

Incremental Update Strategy Rather than full table refreshes, the system implements incremental updates that only recalculate scores for affected geographic areas:

- **Spatial Partitioning**: RUFA scores are partitioned by counties to isolate updates
- **Change Detection**: Modified tree records trigger updates only for their specific census-designated places
- **Dependency Tracking**: A dependency graph ensures that changes to base data propagate correctly through all dependent materialized views

3.5.3 Data Consistency Safeguards

Because refresh is multi-step (read base data, recompute aggregates, write serving tables), readers can otherwise see *transient inconsistency*, namely mismatched rollups across geographic levels, mixed snapshots across partitions, or a schedule table that disagrees with what was published when triggers and batch jobs overlap.

RUFA therefore enforces operational goals of **detectability**, so materialized tables expose whether they are current, in progress, or stale; **atomic publication**, so score tables never appear half-updated during a refresh; and **cascade alignment**, so dependent geographic levels shown together reflect the same logical version of the underlying data.

The Sections below implement this in MariaDB with lightweight metadata, status flags, and atomic table swaps that avoid long-lived locks on the primary serving table.

3.5.3.1 Version Control

Each materialized view maintains version numbers to detect stale data:

Code Listing 3.3: *Version control table for materialized views*

```
CREATE TABLE mv_version_control (
    view_name VARCHAR(50) PRIMARY KEY,
    current_version INT NOT NULL DEFAULT 1,
    last_refresh TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    refresh_duration_seconds INT,
    status ENUM('CURRENT', 'REFRESHING', 'STALE')
       DEFAULT 'CURRENT'
);
```

3.5.3.2 Atomic Updates

Large refresh operations use temporary tables and atomic swaps to prevent serving inconsistent data:

Code Listing 3.4: *Atomic table swap for consistent updates*

```
-- Create temporary table with updated scores
CREATE TABLE mv_rufa_scores_temp LIKE mv_rufa_scores;

-- Populate with fresh calculations
INSERT INTO mv_rufa_scores_temp
SELECT place_id,
       ((ccp_score + td_score + te_score + tpc_score) /
           20) * 100 as rufa_score,
       NOW() as calculated_at
FROM mv_tree_metrics
JOIN mv_population_data USING (place_id);

-- Atomic swap
RENAME TABLE mv_rufa_scores TO mv_rufa_scores_old,
             mv_rufa_scores_temp TO mv_rufa_scores;

DROP TABLE mv_rufa_scores_old;
```

3.5.4 Performance Optimization

The materialized view system includes several performance optimizations:

3.5.4.1 Selective Refresh

Only geographic areas with actual data changes are recalculated, dramatically reducing refresh times from hours to minutes for typical updates.

3.5.4.2 Parallel Processing

Multiple refresh workers can simultaneously update different geographic partitions, leveraging MariaDB's parallel processing capabilities.

3.5.4.3 Caching Strategy

Frequently accessed RUFA scores are cached in Redis with appropriate TTL values, reducing database load for dashboard queries.

3.5.5 Monitoring and Alerting

The system includes comprehensive monitoring to ensure materialized views remain current:

- **Staleness Detection**: Automated alerts when materialized views fall behind source data by more than 12 hours
- **Performance Monitoring**: Tracking of refresh times and resource utilization
- **Data Quality Checks**: Validation that recalculated scores fall within expected ranges

This materialized view architecture ensures that RUFA scores remain accurate and current while providing the query performance necessary for responsive user interactions with the Urban Forestry Assessment Dashboard.

3.5.6 Indexing Results

To evaluate the performance impact of different data storage and indexing strategies, we conducted comprehensive benchmarking tests across various query scenarios. Our analysis compared four different approaches for data retrieval and rendering within the RUFA system, measuring query execution times and system responsiveness under typical usage patterns. All performance tests were conducted on a standardized test environment consisting of an AWS EC2 instance (t3.large) with 2 vCPUs, 8GB RAM, and 10.5.23 MariaDB running on Amazon RDS. The test dataset contained approximately 7.2 million tree records distributed across 702 cities in California, matching the production data scale described in the California Urban Forest Inventory.

Each test scenario was executed 100 times to ensure statistical significance, with query execution times measured using MySQL's built-in profiling tools. The performance metrics were calculated as:

$$\text{Throughput} = \frac{1}{\text{Average Query Time (seconds)}} \tag{3.1}$$

$$\text{Performance Improvement} = \frac{T_{\text{baseline}} - T_{\text{optimized}}}{T_{\text{baseline}}} \times 100\% \tag{3.2}$$

where T_{baseline} represents the baseline query time and $T_{\text{optimized}}$ represents the optimized query time.

3.5.7 Performance Comparison Results

Table 3.1 presents the average query execution times across different storage and indexing strategies for city-specific queries retrieving tree inventory data.

Table 3.1: *Performance comparison of different storage and indexing strategies*

Storage Method	**Avg. Time (ms)**	**Std. Dev.**	**Queries/s**
S3 Single CSV File	8,450	1,230	0.12
S3 Per-City CSV Files	3,120	480	0.32
Database Table (No Index)	2,890	290	0.35
Database Table (Index)	145	25	6.90

3.5.7.1 S3 Single CSV File Approach

The initial approach of storing all tree data in a single CSV file on Amazon S3 demonstrated the poorest performance, with average query times exceeding 8.4 seconds. This method required downloading and parsing the entire dataset for each query, regardless of the specific city or region being requested. The high standard deviation ($\sigma = 1,230$ ms) indicated inconsistent performance, likely due to network latency variations and the overhead of processing large file transfers.

3.5.7.2 S3 Per-City CSV Files

Partitioning the data into individual CSV files per city showed significant improvement, reducing average query times to 3.12 seconds, a 63% performance improvement over the single file approach. This approach eliminated the need to process irrelevant data for city-specific queries. However, the system still faced overhead from file system operations and CSV parsing, limiting its scalability for real-time applications.

3.5.7.3 Database Table Without Indexing

Direct queries against the MySQL database table without proper indexing yielded slightly better performance than the partitioned CSV approach, with average query times of 2.89 seconds. While this eliminated file parsing overhead, the lack of indexing on frequently queried columns (particularly city names and geographic identifiers) required full table scans with $O(n)$ complexity for most operations.

3.5.7.4 Database Table With City Indexing

The implementation of a B-tree index on the city column resulted in dramatic performance improvements, reducing average query times to just 145 milliseconds, a 95% improvement over the non-indexed approach. The low standard deviation ($\sigma = 25$ ms) demonstrates consistent performance across different query patterns. This approach achieved a throughput of 6.90 queries per second, making it suitable for interactive web applications and real-time dashboard updates.

3.5.8 Memory and Storage Impact

The indexing strategy increased database storage requirements by approximately 15%, from 2.3GB to 2.6GB for the complete dataset. However, this storage overhead was offset by significant reductions in memory usage during query execution, as indexed queries required substantially less working memory for result set processing.

The storage overhead can be expressed as:

$$\text{Storage Overhead} = \frac{S_{\text{indexed}} - S_{\text{original}}}{S_{\text{original}}} \times 100\% = \frac{2.6 - 2.3}{2.3} \times 100\% = 13.0\% \quad (3.3)$$

3.5.9 Scaling Considerations

Performance testing with varying dataset sizes demonstrated that the indexed approach maintains logarithmic time complexity $O(\log n)$, ensuring scalability as the urban forest inventory continues to grow. Projections suggest that even with a 10× increase in data volume, indexed query times would remain under 300ms for typical city-based queries, maintaining the system's responsiveness for interactive applications.

Chapter 4

SYSTEM DESIGN

The rapid urban forest assessment system requires a robust and scalable architecture capable of handling large-scale tree inventory data while providing real-time query capabilities for municipal planners and environmental researchers. Building upon the performance analysis presented in Chapter 3, which demonstrated that indexed database queries achieve 95% performance improvements over non-indexed approaches, this Chapter outlines the comprehensive system design that enables efficient processing and visualization of urban forest data across California's 702+ cities. In the overall thesis narrative, this Chapter is supporting deployment context for the database and clustering contributions rather than a primary research result.

The system architecture addresses three critical design requirements: scalability to accommodate the 7.2 million tree records in the California Urban Forest Inventory, performance optimization to support interactive web applications with sub-second response times, and extensibility to incorporate additional environmental datasets and analysis capabilities. The design leverages cloud-native technologies and modern web frameworks to create a maintainable and cost-effective solution.

4.1 System Diagram

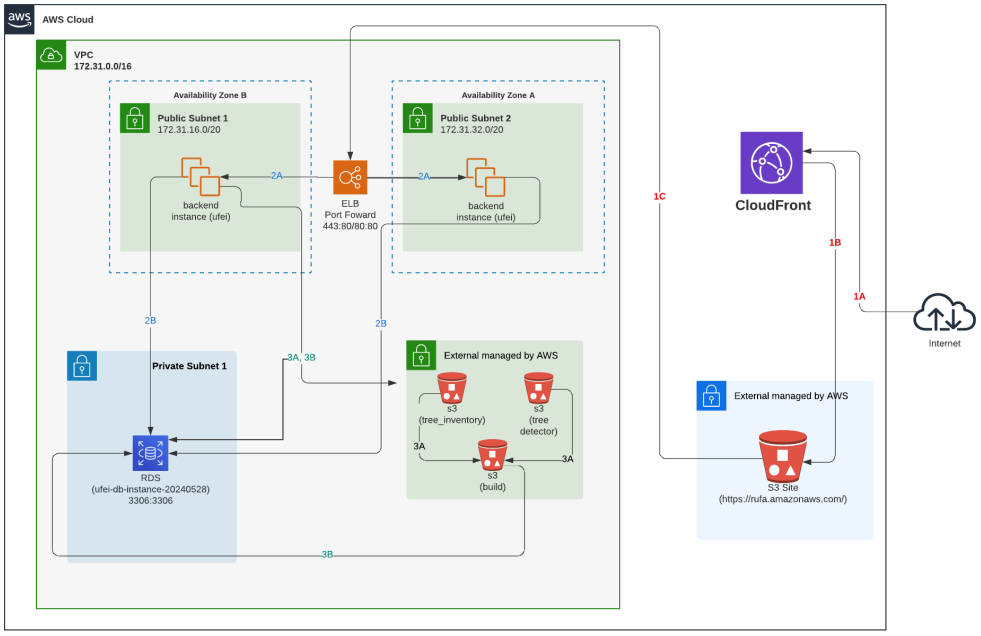


Figure 4.1: *AWS-based system architecture for the RUFA platform.*

Figure 4.1 presents the architecture of the RUFA system, deployed within an AWS Virtual Private Cloud (VPC) for scalability, security, and high availability. The system follows a three-tier design: front-end delivery, backend processing, and data storage.

Table 4.1: *System Component Connections*

Connection	From	To
1A	Users	CloudFront CDN
1B	CloudFront	S3 Static Assets
1C	CloudFront	Backend Services
2A	ELB	EC2 Instances
2B	ELB	HTTPS/HTTP Requests
3A	Backend	RDS MySQL Database
3B	Backend	S3 Data Buckets

Users access the dashboard through CloudFront (1A), which delivers static web assets hosted on Amazon S3 (1B) and routes data requests (1C) to backend services inside the VPC. Backend processing occurs on EC2 instances (2A) distributed across two availability zones, managed by an Elastic Load Balancer (ELB) that forwards HTTPS and HTTP requests (2B). These instances handle user queries, filtering, and aggregation tasks such as computing city-level tree density or generating spatial histograms.

The data layer includes an Amazon RDS MySQL database in a private subnet, securely accessed only by backend instances. Supporting geospatial and inventory datasets are stored in S3 buckets (3A, 3B), which hold tree inventory, canopy detection, and build artifacts. This configuration enables distributed computation on subsets of data for each city or ZIP code.

This architecture enables rapid querying, scalable data processing, and real-time visualization of over seven million urban tree records across California's cities, providing urban planners and environmental researchers with the tools necessary for sustainable planning and evidence-based decision-making.

Chapter 5

CLUSTERING

The clustering component of RUFA groups millions of tree coordinates for interactive map visualization at multiple zoom levels. This Chapter describes the algorithms evaluated during development, from an initial K-means prototype through Super-Clustering with Voronoi tessellation, and the performance trade-offs that motivated each transition.

When RUFA was scoped, UFEI already had a Google BigQuery prototype that clustered tree points as users zoomed. That prototype established the product vision; the task here was to reproduce zoom-aware aggregation inside the MySQL-backed RUFA stack.

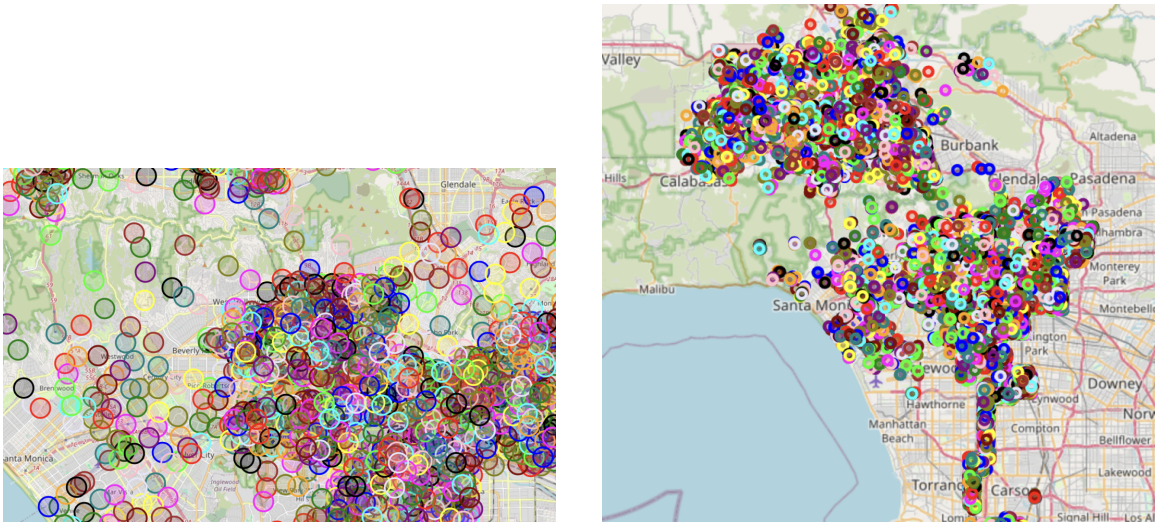
5.1 K-Means

The first implementation applied Lloyd's K-means algorithm [12] to tree coordinates, minimizing the within-cluster sum of squares (WCSS):

$$\text{WCSS} = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2, \tag{5.1}$$

where C_i is cluster i, μ_i its centroid, and x an individual tree location. Centroids were initialized with K-means++ [2], distances were computed with the Haversine formula, and iteration continued until centroid movement fell below 0.001° or 100 iterations were reached.

Because cluster count must be specified in advance, k was chosen dynamically as a function of the number of input points and the map zoom level, typically ranging from 50 to 500 clusters. The intent was to preserve local structure in dense areas while limiting marker overlap at wider views.



(a) *Dynamic k as a function of point count.*

(b) *Insufficient segmentation at city scale.*

Figure 5.1: *K-means clustering of Los Angeles tree coordinates. Scaling k with input size produced overlapping clusters in dense neighborhoods (a) and failed to yield stable multi-scale summaries when the viewport changed (b).*

Figure 5.1 illustrates the limitation on Los Angeles data. Even when k increased with point count, clusters in high-density neighborhoods remained visually indistinguishable, and recomputing assignments for each zoom or pan interaction incurred $O(nkt)$ cost that did not scale to California's full inventory. Benchmarking on 100,000 trees averaged 2.3 seconds per run with 45 MB of working memory. K-means provided a workable baseline but could not meet RUFA's requirement for responsive, zoom-aware aggregation over millions of points.

5.2 Intermediate Approaches

Several refinements were explored before adopting a hierarchical tile-based strategy. Result caching stored centroid positions and cluster assignments across repeated queries, reducing redundant computation when users returned to previously viewed regions, but did not address the fundamental cost of recomputing clusters at each zoom level. Rebuilding a simple greedy radius clustering independently for every zoom level has the same problem: the full dataset would be scanned once per zoom, which is too expensive at statewide scale.

5.3 SuperClustering with Voronoi Diagrams

The production implementation combines SuperCluster, a hierarchical point-clustering library used in web mapping [1], with Voronoi tessellation to define spatial analysis regions at each zoom level. Hierarchical greedy clustering—popularized for interactive maps by Leaflet.markercluster [10]—reuses weighted clusters from finer zoom levels to build coarser ones, so the full point set is not reprocessed for every zoom. SuperCluster precomputes a pyramid of clusters from zoom 0 to 10; at query time, RUFA requests only the clusters intersecting the current bounding box, avoiding full-dataset recomputation on pan and zoom events.

Code Listing 5.1: *SuperCluster initialization and zoom-level processing*

```
index = pysupercluster.SuperCluster(
    detector_points,
    min_zoom=0,
    max_zoom=10,
    radius=5,
    extent=512
)

for zoom in range(5, 10):
```

```
clusters = index.getClusters(bbox, zoom)
if len(clusters) >= 4:
    voronoi_data = process_voronoi(clusters, zoom)
```

For zoom levels with at least four cluster centroids, RUFA generates a Voronoi diagram from the centroid set and converts each cell into a polygon bounded by the viewport. Trees are assigned to cells by point-in-polygon containment, and each cell receives tree density (trees per km^2) and a TD-50 diversity score computed from species counts within the cell (Equation 2.2). Invalid or unbounded Voronoi regions are clipped to the query extent before export as GeoJSON features for the dashboard.

SuperCluster initialization runs in $O(n \log n)$ time; Fortune's sweepline algorithm generates the Voronoi diagram in $O(m \log m)$ for m centroids [7], and point assignment is $O(mn)$ in the naive formulation used during development. In practice, the hierarchical index limits m to the clusters visible at each zoom, and cached tile results make zoom-level switching near instantaneous. Higher zoom levels expose local density variation; lower zoom levels collapse detail into regional summaries suitable for statewide comparison.

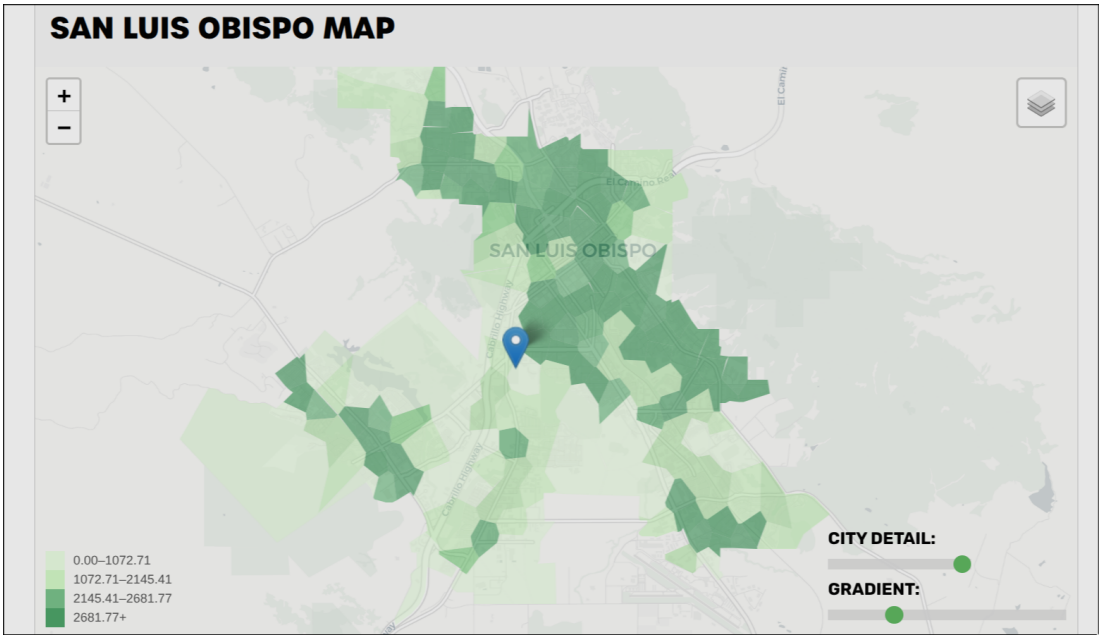


Figure 5.2: *SuperClustering with Voronoi tessellation applied to San Luis Obispo. Each cell summarizes local tree density and species diversity within boundaries that adapt to the underlying point distribution.*

Figure 5.2 shows the final pipeline on San Luis Obispo data. Voronoi cells partition the city into regions aligned with SuperCluster centroids, and color mapping reflects per-cell tree density. This replaces the overlapping K-means markers in Figure 5.1 with a stable, zoom-aware representation suitable for statewide deployment.

Chapter 6

CONCLUSIONS

This thesis addressed two engineering problems encountered in building the Rapid Urban Forest Assessment (RUFA) dashboard: querying and aggregating millions of tree records across hundreds of California census-designated places in real time, and producing stable spatial summaries at multiple zoom levels without recomputing cluster assignments on every map interaction. Each problem was approached by characterizing the workload, prototyping a straightforward solution, measuring where it broke down, and replacing it with an implementation that scales to the production dataset. The RUFA Score formula itself was developed at UFEI; the contribution here is the infrastructure that computes and serves that score and its maps interactively.

On the data side, RUFA's MySQL schema combines normalized inventory and aerial-detection tables with cascading materialized views that propagate RUFA Score metrics from census places to ZIP codes, tracts, and counties. Spatial indexing strategies were evaluated for assigning tree points to administrative polygons, and indexed queries achieved sub-second response times against the full California inventory. The resulting design preserves compatibility with the existing selectree.calpoly.edu infrastructure while delivering the interactive performance required by the dashboard.

On the visualization side, the clustering pipeline evolved from a fixed-k K-means baseline, which produced overlapping markers in dense neighborhoods and recomputed assignments on every zoom event, to a SuperCluster index paired with Voronoi tessellation. The final implementation precomputes a multi-resolution cluster hierarchy

and derives per-cell tree density and TD-50 diversity scores at query time, yielding zoom-aware summaries that match the uneven density of urban tree distributions.

Together with the AWS-based three-tier deployment described in chapter 4, these contributions form a system that aggregates inventory and aerial-detection data, computes the four-component RUFA Score, and serves interactive assessments to urban foresters, planners, and researchers. The composite score reduces a complex, multi-source dataset to a single comparable index, supporting evidence-based decisions about planting, maintenance, and policy across California communities.

6.1 Future Work

Several directions extend RUFA beyond the scope of this thesis.

6.1.1 Allometric Estimation

UFEI's AllomeTree tool [23] exposes species-specific allometric equations for California's urban forest, predicting diameter at breast height (DBH), height, and canopy width with 95% prediction intervals from either DBH or tree age. The equations derive from the U.S. Forest Service Urban Tree Database [15], which fits 365 equation sets across 171 species and 16 climate regions. Integrating these models into RUFA would let the system estimate structural attributes for trees with incomplete inventory records and for the aerial-only detections produced by the Ventura et al. pipeline, which currently provide location and canopy extent but no DBH or height. With those estimates, RUFA could extend its reporting to ecosystem services such as carbon storage, leaf area, and stormwater interception, derived per-tree rather than aggregated from sparse inventory measurements.

6.1.2 Incremental Data Integration

The current materialized-view refresh runs as a batch process. Incremental refresh triggered by inventory updates or new aerial detections would keep RUFA Scores current without recomputing every dependent view, and would support partner organizations contributing inventory data on a rolling basis.

6.1.3 Mobile and Field Interfaces

A field-oriented interface would let urban foresters capture or correct inventory records on site and view RUFA assessments from mobile devices. Bidirectional sync between field captures and the central database would tighten the loop between assessment and on-the-ground management.

6.1.4 Geographic Expansion

Extending RUFA beyond California requires adapting the schema, score normalization, and species references to other climate zones and regulatory contexts. The data model and clustering pipeline are state-agnostic; the main work lies in re-binning each metric against a new statewide distribution and incorporating regional species lists.

As urban areas continue to grow and face increasing environmental pressure, tools that quantify urban forest structure, diversity, and equity will become more important to municipal planning. RUFA contributes to that toolset by combining automated detection, inventory aggregation, and interactive visualization in a single, comparable assessment.

BIBLIOGRAPHY

[1] V. Agafonkin. *Clustering millions of points on a map with Supercluster*. https://blog.mapbox.com/clustering-millions-of-points-on-a-map-with-supercluster-272046ec5c97. Mapbox blog post introducing hierarchical greedy clustering and the Supercluster library. 2016.

[2] D. Arthur and S. Vassilvitskii. "k-means++: The Advantages of Careful Seeding". In: *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM, 2007, pp. 1027–1035.

[3] R. Cattell. "Scalable SQL and NoSQL data stores". In: *ACM SIGMOD Record* 39.4 (2011), pp. 12–27. DOI: 10.1145/1978915.1978919.

[4] E. F. Codd. "A Relational Model of Data for Large Shared Data Banks". In: *Communications of the ACM* 13.6 (1970), pp. 377–387. DOI: 10.1145/362384.362685.

[5] EarthDefine. *TreeMap: High-Resolution Tree Canopy Cover*. https://www.earthdefine.com/treemap/. Accessed 2026. 2018.

[6] R. A. Finkel and J. L. Bentley. "Quad Trees: A Data Structure for Retrieval on Composite Keys". In: *Acta Informatica* 4.1 (1974), pp. 1–9. DOI: 10.1007/BF00288933.

[7] S. Fortune. "A Sweepline Algorithm for Voronoi Diagrams". In: *Algorithmica* 2 (1987), pp. 153–174. DOI: 10.1007/BF01840357.

[8] Green Schoolyards America. *Schoolyard Tree Canopy Equity Study*. https://www.greenschoolyards.org/tree-canopy-equity. California K–12 public school campuses; 6.4% median tree canopy in student zones. Research conducted with GreenInfo Network, 2022–2024. 2024.

[9] A. Guttman. "R-Trees: A Dynamic Index Structure for Spatial Searching". In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1984, pp. 47–57. DOI: 10.1145/602259.602266.

[10] D. Leaver. *Leaflet.MarkerCluster 0.1 Released*. https://leafletjs.com/2012/08/20/guest-post-markerclusterer-0-1-released.html. Guest post on Leaflet introducing hierarchical greedy marker clustering. 2012.

[11] S. J. Livesley, E. G. McPherson, C. Calfapietra, et al. "The Urban Forest and Ecosystem Services: Impacts on Urban Water, Heat, and Pollution Cycles at the Tree, Street, and City Scale". In: *Journal of Urban Ecology* (2016).

[12] S. P. Lloyd. "Least Squares Quantization in PCM". In: *IEEE Transactions on Information Theory* 28.2 (1982), pp. 129–137. DOI: 10.1109/TIT.1982.1056489.

[13] N. L. R. Love and E. G. McPherson. "Diversity and structure in California's urban forest". In: *Urban Forestry & Urban Greening* (2022).

[14] E. G. McPherson, A. M. Berry, S. M. L. Studer, et al. "Performance Testing to Identify Climate-Ready Trees for Central Valley Communities". In: *Arboriculture & Urban Forestry* 44.2 (2018), pp. 83–99. DOI: 10.48044/jauf.2018.008.

[15] E. G. McPherson, N. S. van Doorn, and P. J. Peper. *Urban Tree Database and Allometric Equations*. Gen. Tech. Rep. PSW-GTR-253. Albany, CA: U.S. Department of Agriculture, Forest Service, Pacific Southwest Research Station, 2016, p. 86. DOI: 10.2737/RDS-2016-0005.

[16] G. Niemeyer. *Geohash*. Available at http://geohash.org. 2008.

[17] P. Ramsey and R. Research. *PostGIS Documentation*. Available at https://postgis.net. 2005.

[18] H. Samet. *Foundations of Multidimensional and Metric Data Structures*. Morgan Kaufmann, 2006. ISBN: 9780123694461.

[19] D. J. Sanusi et al. “Street orientation and side of the street greatly influence the microclimatic benefits street trees”. In: *Journal of Environmental Quality* (2016).

[20] State of California. *California Urban Forestry Act of 1978*. https://leginfo.legislature.ca.gov/. Statutory framework for California urban forestry programs; canopy-increase goals referenced in U&CF planning. 1978.

[21] U.S. Census Bureau. *2020 Census of Population and Housing*. https://www.census.gov/programs-surveys/decennial-census/decade/2020/2020-census-main.html. Decennial census population figures used as the TPC denominator in RUFA. 2021.

[22] U.S. Forest Service. *California Urban Tree Canopy Assessment Data*. https://www.fs.usda.gov/detailfull/r5/communityforests/?cid=fseprd647442. Region 5 Community Forests; EarthDefine 2018 product purchased for California.

[23] Urban Forest Ecosystems Institute. *AllomeTree: Allometric Equations for California's Urban Forest*. https://calpolyufei.github.io/allometree-ufei/. Cal Poly Urban Forest Ecosystems Institute.

[24] E. Veach et al. *S2 Geometry Library*. https://s2geometry.io. Open-sourced by Google. Available at https://github.com/google/s2geometry. 2017.

[25] L. Velasquez-Camacho, M. Willaredt, P. Singh, and A. Ossola. “Bleeding green: California's schools are rapidly losing tree canopy cover”. In: *Urban Forestry & Urban Greening* 113 (2025), p. 129117. DOI: 10.1016/j.ufug.2025.129117.

[26] J. Ventura, C. Pawlak, M. Honsberger, et al. “Individual Tree Detection in Large-Scale Urban Environments using High-Resolution Multispectral Imagery”. In: *arXiv* (2022). Available at https://arxiv.org/abs/2208.10607v4.

[27] Q. Xiao and E. G. McPherson. “Surface water storage capacity of twenty tree species in Davis, California. Journal of Environmental Quality”. In: *Journal of Environmental Quality* (2016).